

Extra code snippets for Labs 01-06

Added to the exam reference. Every snippet here was run on test images before inclusion; they are templates, so replace file names and tune

parameters for your image. Variables assumed already loaded: `img`, `gray`, `rgb`, `h`, `w` (page 1), and where named, `ct_gray`, `mri_gray`, `ref_gray`, `sensor`, `pts1`, `pts2`, `corners`, `img1`, `img2`. `show()` (first snippet) is the side-by-side helper used by later ones. Source note: the Lab 01 and Lab 06 manuals are now included below. Lab 02 still follows the task sheet and the announced topics (its manual was unavailable). Where a lab manual's own code does not do what its heading says, the note says so and the last page collects them; verify with your teacher if an exam question quotes the manual.

Lab 01 extras: Python tasks and basic image operations

Everything below assumes the shared setup on page 1 (`img`, `gray`, `rgb`, `h`, `w`). `show()` is the side-by-side helper defined first.

Setup check and side-by-side plotting helper

```
print(cv2.__version__) # validation after install
def show(ims, titles, size=(12,4)): # side-by-side comparison
    plt.figure(figsize=size)
    for i,(im,t) in enumerate(zip(ims,titles),1):
        plt.subplot(1,len(ims),i)
        plt.imshow(im, cmap='gray' if im.ndim==2 else None)
        plt.title(t, fontsize=14, color='darkred', pad=15); plt.axis('off')
    plt.tight_layout(); plt.show()
show([rgb, gray], ['RGB', 'Gray'])
```

Lab 01 says all comparisons use Matplotlib subplots. In Jupyter: `%pip install opencv-python matplotlib` (note the %).

GroceryManager class with error handling

```
class GroceryManager:
    def __init__(self): self.items = {} # name -> {'quantity', 'price'}
    def add_item(self, item, quantity, price):
        if item in self.items: self.items[item]['quantity'] += quantity
        else: self.items[item] = {'quantity': quantity, 'price': price}
    def remove_item(self, item):
        if item not in self.items:
            print(f"Error: '{item}' is not in the list"); return False
        del self.items[item]; return True
    def view_list(self):
        for n, d in self.items.items():
            print(f"{n:<10} qty={d['quantity']:<3} price={d['price']:<6}
                  subtotal={d['quantity']*d['price']}")
    def calculate_total(self):
        return sum(d['quantity']*d['price'] for d in self.items.values())
g = GroceryManager(); g.add_item('milk', 2, 150); g.remove_item('eggs'); g.view_list();
print(g.calculate_total())
```

`remove_item` must refuse a missing item; `calculate_total` = sum of quantity x price.

Nested-dictionary student records: best average and search by major

```
students = {1: {'Name': 'Ali', 'Major': 'AI', 'Grades': [90, 85]},
            2: {'Name': 'Sara', 'Major': 'CS', 'Grades': [95, 92]}}
def top_student(db):
    avg = {k: sum(v['Grades'])/len(v['Grades']) for k, v in db.items() if v['Grades']}
    return db[max(avg, key=avg.get)]['Name']
def by_major(db, major):
    for v in db.values():
        if v['Major'].lower() == major.lower(): print(v['Name'], v['Grades'])
print(top_student(students)); by_major(students, 'ai')
```

Keys: 'Name', 'Major', 'Grades' (list). Guard against empty grade lists.

Target board on a blank canvas with bounding box and centre

```
canvas = np.zeros((800,800,3), np.uint8) # black 800x800
cx, cy = canvas.shape[1]//2, canvas.shape[0]//2 # (400, 400)
cols = [(255,255,255),(0,0,255)]*2 + [(255,255,255)] # alternating, BGR
for r, c in zip([350,280,210,140,70], cols):
    cv2.circle(canvas, (cx,cy), r, c, -1)
cv2.rectangle(canvas, (cx-350,cy-350), (cx+350,cy+350), (0,255,0), 3) # box around outer
circle
```

```
print('Canvas centre:', (cx, cy))
```

Draw the largest filled circle first, otherwise it covers the smaller ones. Thickness -1 fills.

Heavy blur and exact-centre 300x300 ROI (original vs blurred)

```
blur = cv2.GaussianBlur(rgb, (25,25), 0) # heavy blur, odd kernel
cy, cx = h//2, w//2
roi_o = rgb[cy-150:cy+150, cx-150:cx+150] # 300 x 300
roi_b = blur[cy-150:cy+150, cx-150:cx+150]
show([roi_o, roi_b], ['Original ROI', 'Blurred ROI'])
```

ROI is img[y0:y1, x0:x1]; centre = (h/2, w/2) and both ends are 150 away.

Semi-transparent caption bar over the bottom 20% with a title

```
ov = rgb.copy()
cv2.rectangle(ov, (0, int(.8*h)), (w, h), (0,0,255), -1) # solid bar on a COPY
cap = cv2.addWeighted(ov, .5, rgb, .5, 0) # alpha*ov + beta*rgb + gamma
cv2.putText(cap, 'My Title', (20, int(.92*h)), cv2.FONT_HERSHEY_SIMPLEX, 1.5, (255,255,255),
3)
show([rgb, cap], ['Original', 'Caption'])
```

On an RGB array (0,0,255) is blue. Put the text on the blended result so it stays sharp.

Global vs adaptive threshold, then rotate 45 degrees with scale 0.8

```
_ , g = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
a = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11,
2)
M = cv2.getRotationMatrix2D((w/2, h/2), 45, 0.8) # (centre x,y), angle, scale
rot = cv2.warpAffine(a, M, (w, h))
show([g, a, rot], ['Global', 'Adaptive', 'Adaptive rot 45, scale .8'])
```

Scale 0.8 shrinks the rotated image so its corners stay inside the canvas.

Masking with bitwise logic using a filled polygon

```
a = cv2.resize(cv2.imread('image.jpg'), (500,500)); b = cv2.resize(cv2.imread('b.jpg'),
(500,500))
m = np.zeros((500,500), np.uint8)
cv2.fillPoly(m, [np.array([[250,60],[440,200],[370,430],[130,430],[60,200]], np.int32)], 255)
fg = cv2.bitwise_and(a, a, mask=m) # polygon cut out of image A
bg = cv2.bitwise_and(b, b, mask=cv2.bitwise_not(m)) # everything outside from image B
out = cv2.bitwise_or(fg, bg)
```

Both images must be resized to the same size. The mask must be filled (-1 or fillPoly).

Pandas channel profile (Red, Green, Blue columns)

```
import pandas as pd
df = pd.DataFrame({'Red': rgb[:, :, 0].ravel(), 'Green': rgb[:, :, 1].ravel(), 'Blue':
rgb[:, :, 2].ravel()})
print(df.describe()) # count mean std min 25% 50% 75% max
```

Convert to RGB first so the column names are truthful (imread order is B, G, R).

Threshold types, resize modes, halves, wrap vs saturate

```
outs = [cv2.threshold(gray, 127, 255, t)[1] for t in (cv2.THRESH_BINARY,
cv2.THRESH_BINARY_INV,
cv2.THRESH_TRUNC, cv2.THRESH_TOZERO, cv2.THRESH_TOZERO_INV)]
half = cv2.resize(img, None, fx=.5, fy=.5, interpolation=cv2.INTER_AREA) # shrink: INTER_AREA
big = cv2.resize(img, (2*w, 2*h), interpolation=cv2.INTER_CUBIC) # enlarge: CUBIC/LINEAR
left, right = img[:, :w//2], img[:, w//2:]
a, b = np.uint8([[250]]), np.uint8([[10]]) # 1x1 images, not 1-D arrays
wrap = a + b # NumPy wraps -> [[4]]
sat = cv2.add(a, b) # OpenCV saturates -> [[255]]
```

THRESH_BINARY is strictly greater than T and writes maxval (not always 255).

Lab 02 extras: point transforms, histograms, colour, fusion, video

Manual unavailable: these follow the Lab 02 task sheet and the announced topics (log, gamma, piecewise, histogram).

Negative, log (with c), gamma by lookup table, stretch, piecewise, CLAHE

```
neg = 255 - gray # s = L-1-r
f = gray.astype(np.float32)
c = 255/np.log(1 + f.max()); log2 = np.uint8(c*np.log(1 + f)) # s = c*log(1+r), max -> 255
lut_g = np.array([(i/255)**0.5*255 for i in range(256)], np.uint8) # gamma 0.5 brightens
gam = cv2.LUT(gray, lut_g)
stretch = cv2.normalize(gray, None, 0, 255, cv2.NORM_MINMAX) # min-max contrast stretch
```

```
lut_p = np.interp(np.arange(256), [0,70,160,255], [0,30,220,255]).astype(np.uint8) #
    piecewise
pw = cv2.LUT(gray, lut_p)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8)).apply(gray) # local equalization
Cast to float BEFORE log. A lookup table (cv2.LUT) is the fastest way to apply any point transform.
```

Histograms per channel, equalization of a colour image, equalization by hand

```
for i, col in enumerate('bgr'):
    plt.plot(cv2.calcHist([img], [i], None, [256], [0,256]), color=col) # B, G, R histograms
ycc = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
ycc[:, :, 0] = cv2.equalizeHist(ycc[:, :, 0]) # equalize Y only
eq_color = cv2.cvtColor(ycc, cv2.COLOR_YCrCb2BGR)
hist, _ = np.histogram(gray.ravel(), 256, [0,256]); cdf = hist.cumsum()/gray.size
eq_manual = np.uint8(np.round(255*cdf))[gray] # lookup: new = table[old]
Equalize colour images on the luminance channel only. By hand: s = round(255 * CDF(r)).
```

Colour balance (shift and gray-world gain) and dense-tissue mask

```
m = img.reshape(-1,3).mean(0) # channel means [B, G, R]
shift = np.clip(img.astype(np.float32) + (m.mean() - m), 0, 255).astype(np.uint8)
gain = np.clip(img.astype(np.float32) * (m.mean() / m), 0, 255).astype(np.uint8)
eq = cv2.equalizeHist(gray)
_, dense = cv2.threshold(eq, 200, 255, cv2.THRESH_BINARY) # keep only the densest tissue
only_dense = cv2.bitwise_and(eq, eq, mask=dense)
Shift is additive, gain is multiplicative; say which one you used. Clip before converting to uint8.
```

One reusable enhancement function (image or video frame)

```
def enhance(g): # g: uint8 grayscale
    eq = cv2.equalizeHist(g)
    heat = cv2.applyColorMap(eq, cv2.COLORMAP_JET) # 3-channel BGR heatmap
    f = heat.astype(np.float32); m = f.reshape(-1,3).mean(0)
    f = np.clip(f + (m.mean() - m), 0, 255) # colour balance (shift)
    f = 255/np.log(1 + f.max()) * np.log(1 + f) # log: reveals dark regions
    f = 255 * (f/255)**1.5 # gamma > 1: suppresses bright backscatter
    return f.astype(np.uint8)
show([gray, cv2.cvtColor(enhance(gray), cv2.COLOR_BGR2RGB)], ['Raw', 'Enhanced'])
Order from the task: equalize, JET heatmap, colour balance, log, power law. Output is 3-channel uint8.
```

CT + MRI fusion in colour, then log and gamma on the fused matrix

```
mri2 = cv2.resize(mri_gray, (ct_gray.shape[1], ct_gray.shape[0]))
ct_c = cv2.cvtColor(cv2.equalizeHist(ct_gray), cv2.COLOR_GRAY2BGR) # CT stays gray structure
mri_c = cv2.applyColorMap(cv2.equalizeHist(mri2), cv2.COLORMAP_JET) # MRI as heatmap
fz = cv2.addWeighted(ct_c, .7, mri_c, .3, 0).astype(np.float32)
fz = 255/np.log(1 + fz.max()) * np.log(1 + fz) # lift crushed blacks
fz = np.uint8(255 * (fz/255)**1.2) # tame blown whites
show([ct_gray, mri2, cv2.cvtColor(fz, cv2.COLOR_BGR2RGB)], ['CT', 'MRI', 'Fused'])
Same size is mandatory for addWeighted; keep alpha + beta = 1. Heavier weight on CT keeps its sharp edges.
```

Video: raw and enhanced side by side, live and saved

```
cap = cv2.VideoCapture('clip.mp4') # file path (0 would be the webcam)
fps = cap.get(cv2.CAP_PROP_FPS) or 25; W, H = int(cap.get(3)), int(cap.get(4))
out = cv2.VideoWriter('out.mp4', cv2.VideoWriter_fourcc('mp4v'), fps, (2*W, H)) # size =
    (width, height)
while True:
    ok, fr = cap.read()
    if not ok: break # end of video
    both = np.hstack([fr, enhance(cv2.cvtColor(fr, cv2.COLOR_BGR2GRAY))])
    out.write(both)
    # cv2.imshow('raw | enhanced', both)
    # if cv2.waitKey(25) & 0xFF == ord('q'): break
cap.release(); out.release(); cv2.destroyAllWindows()
hstack needs equal height and channels. cv2.imshow may not work in notebooks; save with VideoWriter instead.
```

Lab 03 extras: geometric transformations and the ten task patterns

Column-vector convention: $M = T @ R @ S$ applies S first. `warpAffine` takes the top two rows of a 3×3 .

Hand-built matrices: scale about the centre, shear, translate, reflect

```
cx, cy = w/2, h/2
S3 = np.float32([[3,0,cx-3*cx], [0,3,cy-3*cy]]) # 3x zoom, centre stays put
```

```

zoom = cv2.warpAffine(img, S3, (w,h))
k = .5; shear = cv2.warpAffine(img, np.float32([[1,k,0],[0,1,0]]), (int(w + abs(k)*h), h)) #
x' = x + k*y
tr = cv2.warpAffine(img, np.float32([[1,0,150],[0,1,80]]), (w+150, h+80)) # content moves
right 150, down 80
flip_y = cv2.warpAffine(img, np.float32([[1,0,0],[0,-1,h-1]]), (w,h)) # reflect about the
x-axis

```

Scale about the centre: $3*cx + tx = cx$, so $tx = cx - 3*cx$. Widen the canvas for a shear or translation.

Rotate with a hand-made 2x2 matrix and a canvas big enough for the corners

```

t = np.deg2rad(45); R = np.array([[np.cos(t), -np.sin(t)], [np.sin(t), np.cos(t)]])
nw = int(h*abs(np.sin(t)) + w*abs(np.cos(t))); nh = int(h*abs(np.cos(t)) + w*abs(np.sin(t)))
shift = np.array([nw/2, nh/2]) - R @ np.array([w/2, h/2]) # re-centre on the larger canvas
M = np.hstack([R, shift[:,None]]).astype(np.float32)
rot = cv2.warpAffine(img, M, (nw, nh))

```

New size = $(h|sin|+w|cos|, h|cos|+w|sin|)$. With y pointing down, +theta here turns the picture CLOCKWISE;
getRotationMatrix2D(deg) is
counter-clockwise.

3x3 builders: rigid and similarity in one matrix

```

def T(tx, ty): return np.float32([[1,0,tx],[0,1,ty],[0,0,1]])
def Sc(s): return np.diag([s, s, 1]).astype(np.float32)
def Rot(d, cx=0, cy=0):
    a = np.deg2rad(d); c, s = np.cos(a), np.sin(a)
    return T(cx,cy) @ np.float32([[c,-s,0],[s,c,0],[0,0,1]]) @ T(-cx,-cy)
M_rigid = T(100,50) @ Rot(30, w/2, h/2) # rotate, then translate
M_sim = T(100,50) @ Rot(30) @ Sc(.5) # scale, rotate, translate
out = cv2.warpAffine(img, M_sim[:2], (w,h))

```

Rigid = rotation + translation (scale 1). Similarity adds one uniform scale. Rotation about a point = $T(c) R T(-c)$.

General affine from 3 landmark pairs (solve the 6 unknowns), invert, homography from many points

```

src = np.float32([[50,50],[300,60],[80,250]]); dst = np.float32([[40,70],[310,40],[100,270]])
A = np.hstack([src, np.ones((3,1), np.float32)]) # rows [x y 1]
M = np.linalg.solve(A, dst).T.astype(np.float32) # 2x3 = [[a,b,tx],[c,d,ty]]
fixed = cv2.warpAffine(img, M, (w,h)) # same as cv2.getAffineTransform(src, dst)
Minv = cv2.invertAffineTransform(M)
H, inl = cv2.findHomography(pts2, pts1, cv2.RANSAC, 5.0) # more than 4 noisy matches

```

Each pair gives $x' = ax + by + tx$ and $y' = cx + dy + ty$. M must map glitched -> original; invert if you have the reverse.

Panorama canvas and bird's-eye correction

```

H = cv2.getPerspectiveTransform(pts2, pts1) # image-2 points -> image-1 points (float32, 4x2)
pano = cv2.warpPerspective(img2, H, (w1 + w2, max(h1, h2))); pano[0:h1, 0:w1] = img1
S = 500; sq = np.float32([[0,0],[S,0],[S,S],[0,S]])
top = cv2.warpPerspective(img, cv2.getPerspectiveTransform(corners, sq), (S,S)) # perfect
square view

```

Keep the corner order identical in src and dst (TL, TR, BR, BL). Four pairs fix the 8 unknowns of H.

Paste a painting into an angled frame (scale, rigid, projective composed once)

```

art = cv2.imread('b.jpg'); ph, pw = art.shape[:2]; fh, fw = img.shape[:2]
Rg = T(60,40) @ Rot(10) @ Sc(.5) # linear scale + rigid move
cor = np.float32([[0,0],[pw,0],[pw,ph],[0,ph]]).reshape(-1,1,2)
near = cv2.perspectiveTransform(cor, Rg).reshape(4,2) # painting corners after scale + rigid
frame_pts = np.float32([[120,90],[420,60],[450,330],[100,300]]) # frame's 4 inner corners
Htot = cv2.getPerspectiveTransform(near, frame_pts) @ Rg
warped = cv2.warpPerspective(art, Htot, (fw,fh))
mask = cv2.warpPerspective(np.full((ph,pw), 255, np.uint8), Htot, (fw,fh))
comp = img.copy(); comp[mask > 0] = warped[mask > 0]

```

Compose first, warp once (one interpolation). Warp a white mask the same way to paste only inside the quad.

Lab 04 extras: descriptors, kernels, convolution modes, edges

skimage provides hog and local_binary_pattern; OpenCV provides everything else here.

HOG and uniform LBP with scikit-image

```

from skimage.feature import hog, local_binary_pattern
from skimage import exposure
feat, hog_img = hog(gray, pixels_per_cell=(8,8), cells_per_block=(2,2), visualize=True)
hog_vis = exposure.rescale_intensity(hog_img, in_range=(0,10)) # brighten for display
P, R = 8, 1 # P = 8*radius neighbours

```

```
lbp = local_binary_pattern(gray, P, R, method='uniform')
hist, _ = np.histogram(lbp.ravel(), bins=np.arange(0, P+3), range=(0, P+2)) # 10 bins
hist = hist/(hist.sum() + 1e-6) # normalise
```

HOG length = (cells_y-1)(cells_x-1) x 2x2 x 9 (64x128 window gives 3780). Uniform LBP gives P+2 bins.

Histogram of edge directions, intensity gradients and colour (corrected)

```
sm = cv2.GaussianBlur(gray, (5,5), 0); edges = cv2.Canny(sm, 30, 70)
gx = cv2.Sobel(sm, cv2.CV_64F, 1, 0, ksize=3); gy = cv2.Sobel(sm, cv2.CV_64F, 0, 1, ksize=3)
ang = np.degrees(np.arctan2(gy, gx)) % 360
hed, _ = np.histogram(ang[edges > 0], bins=8, range=(0,360)) # 8 bins of 45 degrees
mag = np.hypot(gx, gy)
hig, _ = np.histogram(ang % 180, bins=9, range=(0,180), weights=mag); hig = hig/hig.sum()
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV) # H range is 0-179
ch = cv2.calcHist([hsv], [0,1,2], None, [8,8,8], [0,180,0,256,0,256]); ch = cv2.normalize(ch,
ch).flatten()
```

arctan2 returns -180..180, so wrap with % 360 or negative angles are dropped. HED uses edge pixels only.

Texture energy and contrast with local windows

```
f = gray.astype(np.float32); k = (3,3)
energy = cv2.boxFilter(f*f, -1, k, normalize=False) # sum of squares in window
contrast = np.sqrt(np.maximum(cv2.blur(f*f, k) - cv2.blur(f, k)**2, 0)) # sqrt(E[x^2] - E[x]^2)
eh, _ = np.histogram(energy, bins=256, range=(0, energy.max())); eh = eh/eh.sum()
ch2, _ = np.histogram(contrast, bins=256, range=(0, contrast.max())); ch2 = ch2/ch2.sum()
```

Cast to float32 first (uint8 squared wraps). Energy = local sum of squares; contrast = local standard deviation.

Kernels: box, 2-D Gaussian, Sobel, Scharr, Prewitt, emboss

```
box = cv2.filter2D(gray, -1, np.ones((3,3), np.float32)/9)
g1 = cv2.getGaussianKernel(5, 1.0); gau = cv2.filter2D(gray, -1, g1 @ g1.T) # g1 alone blurs
one direction
sx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3); sy = cv2.Sobel(gray, cv2.CV_64F, 0, 1,
ksize=3)
sob = np.uint8(np.clip(np.hypot(sx, sy), 0, 255)) # magnitude, then to uint8
scharr_x = cv2.Scharr(gray, cv2.CV_64F, 1, 0)
prewitt_x = cv2.filter2D(gray, cv2.CV_64F, np.float32([[-1,0,1],[-1,0,1],[-1,0,1]]))
emboss = cv2.filter2D(gray, -1, np.float32([[-2,-1,0],[-1,1,1],[0,1,2]]))
direction = np.arctan2(sy, sx) # gradient direction (radians)
```

getGaussianKernel returns a 5x1 column; make it 2-D with g @ g.T (or use GaussianBlur). Use CV_64F to keep signs.

Same, valid and strided convolution with filter2D

```
K = np.float32([[1,0,-1],[2,0,-2],[1,0,-1]]); p = K.shape[0]//2
same = cv2.filter2D(gray, cv2.CV_32F, K, borderType=cv2.BORDER_CONSTANT) # zero padding, same
size
valid = same[p:-p, p:-p] # no padding: (H-K+1, W-K+1)
stride2 = same[:,::2, ::2] # stride 2 by slicing
print(gray.shape, valid.shape, stride2.shape)
```

filter2D has no 'valid' mode and no strides argument. Output size = floor((N + 2P - K)/S) + 1.

Laplacian of Gaussian with zero-crossings, Harris and Shi-Tomasi corners

```
bl = cv2.GaussianBlur(gray, (5,5), 1.4); lap = cv2.Laplacian(bl, cv2.CV_64F)
sg = np.sign(lap); zc = np.zeros(gray.shape, np.uint8)
zc[:, :-1] |= (sg[:, :-1]*sg[:, 1:] < 0).astype(np.uint8) # sign change left-right
zc[:, :-1, :] |= (sg[:, :-1, :]*sg[:, 1:, :] < 0).astype(np.uint8) # sign change up-down
log_edges = zc*255 # also threshold |lap| to cut noise
lap_abs = cv2.convertScaleAbs(lap) # |Laplacian| as uint8
harris = cv2.cornerHarris(np.float32(gray), 2, 3, 0.04)
cimg = img.copy(); cimg[harris > .01*harris.max()] = (0,0,255)
pts = cv2.goodFeaturesToTrack(gray, 100, 0.01, 10) # maxCorners, quality, minDistance
```

First derivative: edge = peak (Sobel, Canny). Second derivative: edge = zero-crossing (LoG). Corners are not in the manual body.

Lab 05 extras: segmentation experiments from the task sheet

Record the parameters you chose and justify them; save outputs with cv2.imwrite('output/name.png', result).

Three global thresholds, six adaptive results, Otsu with its histogram

```
glob = [cv2.threshold(gray, t, 255, cv2.THRESH_BINARY)[1] for t in (80, 127, 170)]
grid = [cv2.adaptiveThreshold(gray, 255, m, cv2.THRESH_BINARY, b, C)
for m in (cv2.ADAPTIVE_THRESH_MEAN_C, cv2.ADAPTIVE_THRESH_GAUSSIAN_C)
```

```

    for b, C in ((11,2), (31,10), (71,20))] # 6 adaptive results
    T, otsu = cv2.threshold(cv2.GaussianBlur(gray, (5,5), 0), 0, 255, cv2.THRESH_BINARY +
        cv2.THRESH_OTSU)
    print('Otsu T =', T) # T comes back first
    plt.hist(gray.ravel(), 256, [0,256]); plt.axvline(T, color='r')
    T = local mean (or Gaussian-weighted mean) minus C. Bigger C lowers T: cleaner background, thin strokes may fade.

```

HSV masks: too restrictive vs final, and red (hue wraps around 0)

```

hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
tight = cv2.inRange(hsv, (24,200,200), (27,255,255)) # too restrictive: holes
final = cv2.inRange(hsv, (20,100,100), (35,255,255)) # yellow, more complete
red = cv2.inRange(hsv, (0,100,100), (10,255,255)) | cv2.inRange(hsv, (170,100,100),
    (179,255,255))
extracted = cv2.bitwise_and(img, img, mask=final)

```

Narrow S and V bounds miss highlights and shadows, leaving holes; hue carries the colour identity.

Canny with three threshold pairs

```

bl = cv2.GaussianBlur(gray, (5,5), 1.4)
res = [cv2.Canny(bl, lo, hi) for lo, hi in ((50,100), (50,200), (150,250))]
show([gray] + res, ['Original', '50/100', '50/200', '150/250'])

```

Raise the high threshold with low fixed: fewer strong seeds, more gaps. Low thresholds admit noise edges.

Region growing (manual stack version): seed and threshold study

```

def region_growing(image, seed, thr):
    mask = np.zeros_like(image, np.uint8); stack = [seed]; s0 = int(image[seed])
    while stack:
        x, y = stack.pop()
        if x < 0 or x >= image.shape[0] or y < 0 or y >= image.shape[1]: continue
        if mask[x, y] == 0 and abs(int(image[x, y]) - s0) < thr: # int() avoids uint8 wrap
            mask[x, y] = 255; stack.extend([(x+1,y), (x-1,y), (x,y+1), (x,y-1)])
    return mask
masks = [region_growing(gray, s, t) for s in ((h//2, w//2), (10, 10)) for t in (10, 25, 50)]

```

Seed is (row, col); the test is against the seed intensity. Too high a threshold leaks, too low fragments.

Watershed with marker count and a distance-threshold sweep (Task 8 table)

```

_, th = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
k = np.ones((5,5), np.uint8); op = cv2.morphologyEx(th, cv2.MORPH_OPEN, k, iterations=2)
sure_bg = cv2.dilate(op, k, iterations=3); dist = cv2.distanceTransform(op, cv2.DIST_L2, 5)
rows = []
for frac in (.2, .4, .6, .8): # distance-transform cutoff
    _, fg = cv2.threshold(dist, frac*dist.max(), 255, 0); fg = np.uint8(fg)
    n, mk = cv2.connectedComponents(fg); mk += 1; mk[cv2.subtract(sure_bg, fg) == 255] = 0
    mk = cv2.watershed(img.copy(), mk) # img: 8-bit, 3-channel
    rows.append((frac, n - 1, len(np.unique(mk)) - 2)) # (cutoff, markers, regions)
print(rows)
out = img.copy(); out[mk == -1] = (0,0,255) # red boundaries (BGR)

```

Dark objects on light: THRESH_BINARY_INV + OTSU. Markers + 1 so background is not 0. Boundary pixels are -1.

K-means for K = 2, 4, 6

```

Z = np.float32(img.reshape(-1,3)); crit = (cv2.TERM_CRITERIA_EPS +
    cv2.TERM_CRITERIA_MAX_ITER, 100, .2)
outs = {}
for K in (2, 4, 6):
    _, lab, cen = cv2.kmeans(Z, K, None, crit, 10, cv2.KMEANS_RANDOM_CENTERS)
    outs[K] = np.uint8(cen)[lab.flatten()].reshape(img.shape) # at most K distinct colours
show([rgb] + [cv2.cvtColor(outs[K], cv2.COLOR_BGR2RGB) for K in (2,4,6)],
    ['Original', 'K=2', 'K=4', 'K=6'])

```

Input must be float32; labels are (N,1) so flatten before indexing the centres. attempts=10 restarts the clustering.

Lab 06 extras: wavelets, boundaries, Hough, SIFT and the lab scenarios

pywt is PyWavelets (pip install PyWavelets). The screen and security snippets are suggested designs; the manual gives no code for them.

Wavelet denoising with residual anomalies, and a 2-D DWT

```

import pywt
x = np.asarray(sensor, float) # 1-D sensor data
co = pywt.wavedec(x, 'db4', level=4) # [ca4, cd4, cd3, cd2, cd1]

```

```
sig = np.median(np.abs(co[-1]))/0.6745; thr = sig*np.sqrt(2*np.log(len(x))) # universal
threshold
co[1:] = [pywt.threshold(c, thr, mode='soft') for c in co[1:]]
den = pywt.waverec(co, 'db4')[:len(x)]; res = x - den # waverec can be 1 sample longer
anom = np.abs(res) > 2*res.std() # tune the multiplier
plt.subplot(211); plt.plot(x); plt.plot(den, '--')
plt.subplot(212); plt.plot(res, 'r'); plt.plot(np.where(anom)[0], res[anom], 'go')
cA, (cH, cV, cD) = pywt.dwt2(gray, 'haar') # 2-D: approximation +
horizontal/vertical/diagonal
```

Noise lives in small detail coefficients: threshold them, reconstruct, then residual = signal - denoised.

Polar HoughLines (rho, theta) drawn as long lines

```
edges = cv2.Canny(cv2.GaussianBlur(gray, (5,5), 0), 50, 150)
lines = cv2.HoughLines(edges, 1, np.pi/180, 150) # rho resolution, theta resolution, votes
v = img.copy()
if lines is not None:
    for rho, theta in lines[:,0]:
        a, b = np.cos(theta), np.sin(theta); x0, y0 = a*rho, b*rho
        cv2.line(v, (int(x0 - 1000*b), int(y0 + 1000*a)), (int(x0 + 1000*b), int(y0 -
        1000*a)), (0,0,255), 2)
```

$\rho = x \cos(\theta) + y \sin(\theta)$. Use polar form because vertical lines have infinite slope. Result may be None.

Lane detection: Canny, road-shaped ROI, HoughLinesP, left/right by slope

```
edges = cv2.Canny(cv2.GaussianBlur(gray, (5,5), 0), 50, 150)
roi = np.zeros_like(edges)
cv2.fillPoly(roi, [np.array([(0,h), (w//2-50,h//2+50), (w//2+50,h//2+50), (w,h)]),
np.int32]], 255)
segs = cv2.HoughLinesP(cv2.bitwise_and(edges, roi), 1, np.pi/180, 50, minLineLength=40,
maxLineGap=100)
ov = np.zeros_like(img); left, right = [], []
for x1, y1, x2, y2 in (segs[:,0] if segs is not None else []):
    s = (y2 - y1)/(x2 - x1 + 1e-6); (left if s < 0 else right).append((x1,y1,x2,y2))
    cv2.line(ov, (x1,y1), (x2,y2), (0,255,0), 3)
lane = cv2.addWeighted(img, .8, ov, 1, 0)
```

In image coordinates y points down, so the left lane has negative slope. The ROI removes buildings and sky.

Count coins with HoughCircles and study param2

```
soft = cv2.medianBlur(gray, 5)
for p2 in (20, 30, 40):
    c = cv2.HoughCircles(soft, cv2.HOUGH_GRADIENT, 1.2, 30, param1=100, param2=p2,
minRadius=10, maxRadius=80)
    print('param2 =', p2, 'coins =', 0 if c is None else c.shape[1])
```

Result shape is (1, N, 3) or None. Lower param2 gives more circles (including false ones); blur first.

SIFT keypoints and a reusable matcher (ratio test)

```
sift = cv2.SIFT_create(); kp, des = sift.detectAndCompute(gray, None); print(len(kp),
des.shape)
vis = cv2.drawKeypoints(img, kp, None, flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
def sift_match(a, b, ratio=.75): # a, b: grayscale images
    s = cv2.SIFT_create(); ka, da = s.detectAndCompute(a, None); kb, db =
    s.detectAndCompute(b, None)
    m = cv2.BFMatcher(cv2.NORM_L2).knnMatch(da, db, k=2)
    return ka, kb, [p[0] for p in m if len(p) == 2 and p[0].distance < ratio*p[1].distance]
```

Descriptors are (N,128) float32. queryIdx indexes the first descriptor set, trainIdx the second.

Panorama stitching with SIFT and a homography

```
g1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY); g2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
k1, k2, good = sift_match(g1, g2)
p2 = np.float32([k2[m.trainIdx].pt for m in good]).reshape(-1,1,2)
p1 = np.float32([k1[m.queryIdx].pt for m in good]).reshape(-1,1,2)
H, _ = cv2.findHomography(p2, p1, cv2.RANSAC, 5.0) # needs at least 4 good matches
pano = cv2.warpPerspective(img2, H, (img1.shape[1] + img2.shape[1], img1.shape[0]))
pano[:img1.shape[0], :img1.shape[1]] = img1
```

findHomography(src, dst) maps src to dst, so for 'image 2 into image 1' src = image-2 points.

Object recognition in video: reference computed once, matched every frame

```
sift = cv2.SIFT_create(); bf = cv2.BFMatcher(cv2.NORM_L2)
kr, dr = sift.detectAndCompute(ref_gray, None); rh, rw = ref_gray.shape
cap = cv2.VideoCapture('clip.mp4')
```

```

while True:
    ok, fr = cap.read()
    if not ok: break
    kf, df = sift.detectAndCompute(cv2.cvtColor(fr, cv2.COLOR_BGR2GRAY), None)
    if df is None or len(df) < 2: continue
    good = [p[0] for p in bf.knnMatch(dr, df, k=2) if len(p) == 2 and p[0].distance <
        .75*p[1].distance]
    if len(good) >= 10:
        s = np.float32([kr[m.queryIdx].pt for m in good]).reshape(-1,1,2)
        d = np.float32([kf[m.trainIdx].pt for m in good]).reshape(-1,1,2)
        H, _ = cv2.findHomography(s, d, cv2.RANSAC, 5.0)
        if H is not None:
            box =
                cv2.perspectiveTransform(np.float32([[0,0],[rw,0],[rw,rh],[0,rh]]).reshape(-1,1,
                    2),
                    H)
            cv2.polylines(fr, [np.int32(box)], True, (0,255,0), 3) # bounding box on the
                frame
    cap.release()

```

Compute the reference keypoints outside the loop. Require enough good matches (e.g. 10) before the homography.

Screen detection with Hough lines (suggested design)

```

segs = cv2.HoughLinesP(edges, 1, np.pi/180, 60, minLineLength=40, maxLineGap=10)
mk = np.zeros_like(gray)
for x1, y1, x2, y2 in (segs[:,0] if segs is not None else []):
    ang = abs(np.degrees(np.arctan2(y2 - y1, x2 - x1)))
    if ang < 10 or abs(ang - 90) < 10 or ang > 170: cv2.line(mk, (x1,y1), (x2,y2), 255, 3)
cnts, _ = cv2.findContours(mk, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
screens = []
for c in cnts:
    ap = cv2.approxPolyDP(c, .02*cv2.arcLength(c, True), True)
    if len(ap) == 4 and cv2.contourArea(ap) > 2000: # 4 corners, plausible size
        x, y, bw, bh = cv2.boundingRect(ap); screens.append((x, y, bw, bh, gray[y:y+bh,
            x:x+bw].mean() > 80))
print('screens found:', len(screens)) # last item True = on

```

Keep near-horizontal and near-vertical segments, form rectangles, decide on/off by mean brightness. Fewer rectangles than expected = missing screen.

Security zone alarm with background subtraction (suggested design)

```

zone = np.array([[100,100], [400,100], [400,300], [100,300]], np.int32)
bs = cv2.createBackgroundSubtractorMOG2()
cap = cv2.VideoCapture('clip.mp4')
while True:
    ok, frame = cap.read()
    if not ok: break
    fg = cv2.morphologyEx(bs.apply(frame), cv2.MORPH_OPEN, np.ones((5,5), np.uint8))
    cnts, _ = cv2.findContours(fg, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    cv2.polylines(frame, [zone], True, (255,0,0), 2)
    for c in cnts:
        if cv2.contourArea(c) > 500:
            x, y, bw, bh = cv2.boundingRect(c)
            if cv2.pointPolygonTest(zone, (x + bw//2, y + bh//2), False) >= 0:
                cv2.rectangle(frame, (x,y), (x+bw, y+bh), (0,0,255), 2)
                cv2.putText(frame, 'ALARM', (20,40), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255),
                    3)
    cap.release()

```

pointPolygonTest returns ≥ 0 for a point inside or on the polygon. The alarm here is on-screen text; add a beep if required.

Parameter and formula quick reference

Function Arguments and what they do

cv2.threshold (src, thresh, maxval, type) returns (T, dst). BINARY: dst = maxval if src > thresh else 0. Add THRESH_OTSU and pass thresh=0 to auto-select.

cv2.adaptiveThreshold (src, maxValue, method MEAN_C | GAUSSIAN_C, type, blockSize odd > 1, C). T(x,y) = local mean (or Gaussian-weighted mean) minus C. Single-channel uint8 only.

cv2.Canny (img, low, high, apertureSize=3, L2gradient=False). Above high = strong; between = kept only if linked to strong; below low = dropped. High about 2 to 3 times low.

cv2.Sobel (src, ddepth, dx, dy, ksize). dx=1,dy=0 finds vertical edges. Use CV_64F to keep negative values;

magnitude = $\sqrt{gx^2 + gy^2}$.

cv2.GaussianBlur (src, (k,k), sigmaX). k positive and odd. sigmaX=0 derives sigma from k. Larger k gives stronger blur.

cv2.getRotationMatrix2D (center (x,y), angle degrees, scale) returns 2x3. Positive angle = counter-clockwise. Pair with warpAffine(img, M, (w,h)).

cv2.getAffineTransform / 3 point pairs (float32) give a 2x3 affine; 4 pairs give a 3x3 homography. Same corner order in src and dst.

getPerspectiveTransform findHomography(src, dst, cv2.RANSAC, 5.0) for many noisy pairs.

cv2.calcHist ([img], [channel], mask, [bins], [min, max]). HSV hue range is [0,180]; use [0,180,0,256,0,256] for a 3-D HSV histogram.

cv2.distanceTransform (binary, cv2.DIST_L2, maskSize 5). Each white pixel gets its distance to the nearest black pixel; cores of

objects are the peaks.

cv2.watershed (img 8-bit 3-channel, markers int32). Background marker must not be 0 (0 = unknown). Result: boundaries = -1.

cv2.kmeans (float32 data (N,3), K, None, criteria, attempts, flags). Returns (compactness, labels (N,1), centers (K,3)).

Rebuild with centers[labels.flatten()].

cv2.HoughLines / HoughLinesP (edges, rho, theta, threshold). P version adds minLineLength, maxLineGap and returns [x1,y1,x2,y2]. Both

can return None.

cv2.HoughCircles (gray, HOUGH_GRADIENT, dp, minDist, param1 = upper Canny, param2 = centre votes, minRadius, maxRadius). Lower param2 = more circles. Returns (1,N,3) or None.

SIFT + BFMatcher SIFT_create().detectAndCompute(gray, None) gives keypoints and (N,128) descriptors.

BFMatcher(NORM_L2).knnMatch(d1, d2, k=2); keep m.distance < 0.75 * n.distance; RANSAC inliers;

need at least 4 pairs for a homography.

skimage hog / LBP hog(gray, pixels_per_cell, cells_per_block, visualize=True). local_binary_pattern(gray, P, R, method="uniform") has P + 2 labels.

cv2.VideoCapture cap.read() returns (ok, frame); ok False ends the loop. waitKey(25) & 0xFF == ord("q") quits. Always release(). VideoWriter(path, fourcc, fps, (width, height)).

Formulas and counts worth having on the page

Topic Formula / fact

Negative s = 255 - r

Log s = $c * \log(1 + r)$, $c = 255 / \log(1 + r_{\max})$. Expands dark, compresses bright. Cast to float first.

Gamma s = $255 * (r/255)^\gamma$. Gamma < 1 brightens (expands darks); gamma > 1 darkens.

Min-max stretch s = $(r - r_{\min}) / (r_{\max} - r_{\min}) * 255$

Piecewise slopes $s1/r1$, $(s2 - s1)/(r2 - r1)$, $(255 - s2)/(255 - r2)$. Slope > 1 stretches, slope < 1 compresses.

Equalization $s_k = (L - 1) * CDF(k) = (L - 1) * \sum_{j \leq k} p(r_j)$ for j <= k. Single-channel uint8 only.

Blend dst = alpha * src1 + beta * src2 + gamma. Keep alpha + beta = 1 to preserve brightness.

Affine / homography Affine: $x' = ax + by + tx$, 6 unknowns, 3 pairs. Homography: $x' = (h11x + h12y + h13)/(h31x + h32y + h33)$,

8 unknowns, 4 pairs.

Degrees of freedom Translation 2, rigid 3, similarity 4, affine 6, projective 8.

Rotation bounding box new_w = $h|\sin t| + w|\cos t|$; new_h = $h|\cos t| + w|\sin t|$; shift M[:,2] by (new_w/2 - w/2, new_h/2 - h/2).

Convolution size $\text{floor}((N + 2P - K)/S) + 1$. Valid: $N - K + 1$. Same: $P = (K - 1)/2$.

HOG length (cells_y - 1)(cells_x - 1) * 2 * 2 * 9. A 64x128 window gives 3780.

LBP 8 neighbours: neighbour >= centre gives 1. Uniform LBP with P = 8 has 10 bins (P + 2); plain LBP has 256.

Otsu Chooses T maximizing between-class variance $w0 * w1 * (\mu0 - \mu1)^2$; best for bimodal histograms.

Hough line rho = x cos(theta) + y sin(theta). Circle: $(x - a)^2 + (y - b)^2 = r^2$, 3-D accumulator (a, b, r).

SIFT DoG = L(k sigma) - L(sigma); extremum compared with 26 neighbours (8 + 9 + 9); descriptor 4x4 cells x 8 bins = 128 values.

Wavelet denoise sigma = median(|cD1|)/0.6745; threshold = sigma * sqrt(2 ln n); soft-threshold details; reconstruct; residual = signal - denoised.

Manual caveats collected (my reading; confirm with your teacher)

- HED/HIG: arctan2 returns -180 to 180, so the manual's histogram over range (0,360) drops negative angles; its HED histogram also

counts all pixels, not only edge pixels.

- Texture energy: image**2 on a uint8 array wraps; the manual's "contrast" filter2D(image,-1,ones) is a local sum, not a standard deviation.

- filter2D has no strides argument, no true "valid" mode, and getGaussianKernel returns a 5x1 column (use g @ g.T for 2-D).

- Watershed: the manual draws [255,0,0] on a BGR image (blue); matplotlib then swaps channels. Convert before plt.imshow.

- Lab 1 putText: the text says scale 1.5 but the code uses 5; threshold example writes maxval 200, not 255.

- LoG: the manual only displays |Laplacian|; real edges are the zero-crossings.